

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №5
по дисциплине «Анализ, моделирование и оптимизация систем»
Тема: Структурная оптимизация многопроцессорной системы
обработки данных.

Студент гр. 1306

Шаврин А.П.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2025

Цель работы.

Целью работы является освоение навыков распределения нагрузки в многопроцессорной вычислительной системе.

Для достижения поставленной цели требуется решить следующие задачи:

- 1) Определить основные характеристики модели
- 2) Провести анализ загрузки многопроцессорной системы.

Постановка задачи.

1. Определить оптимальное число процессов $K_{пр}$ для МПСОД, граф решения ЗНЗ которой приведен на рис 1.

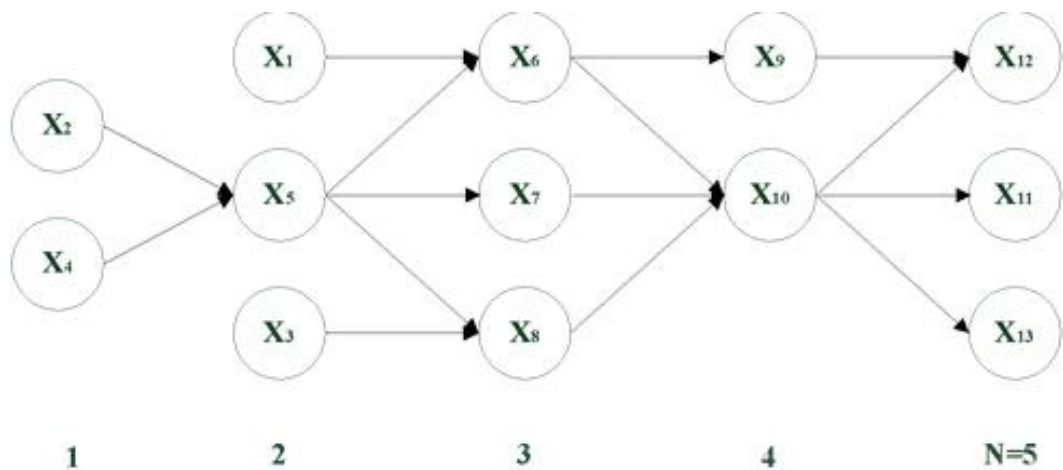


Рис 1. Граф решения ЗНЗ

2. Построить график загрузки каждого процессора, чтобы достигнуть значения $T_{кр}$.

Описание последовательности выполнения работы:

Выполнение работы.

1. Зададим самостоятельно время выполнения каждой задачи для каждого процессора (узла в графе), представленного на рис. 2

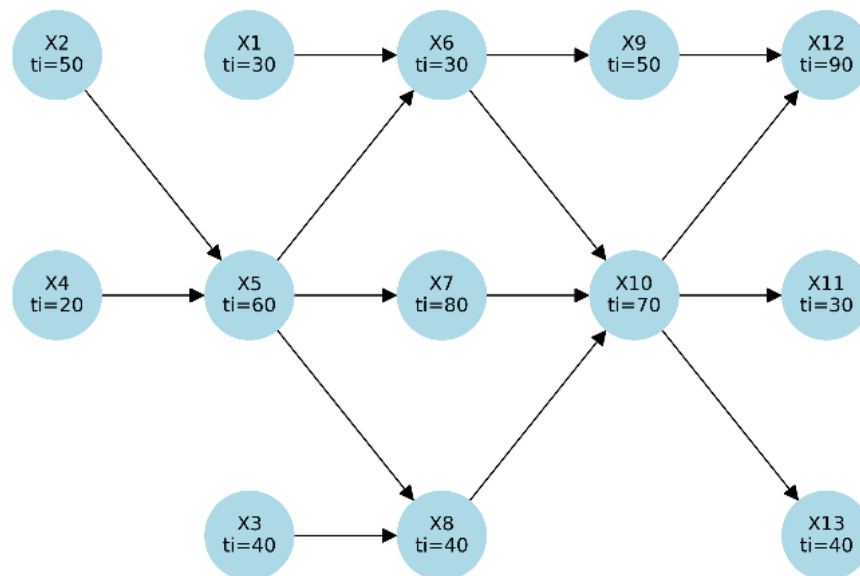


Рис 2. Исходный граф

2. Определим критическое значение $T_{кр}$ (ограничение времени выполнения сверху), результат на рис 3.

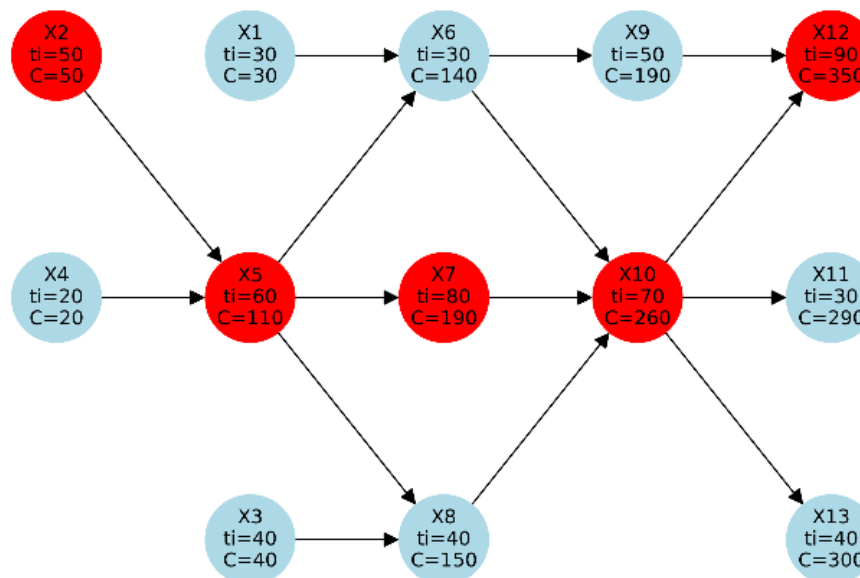
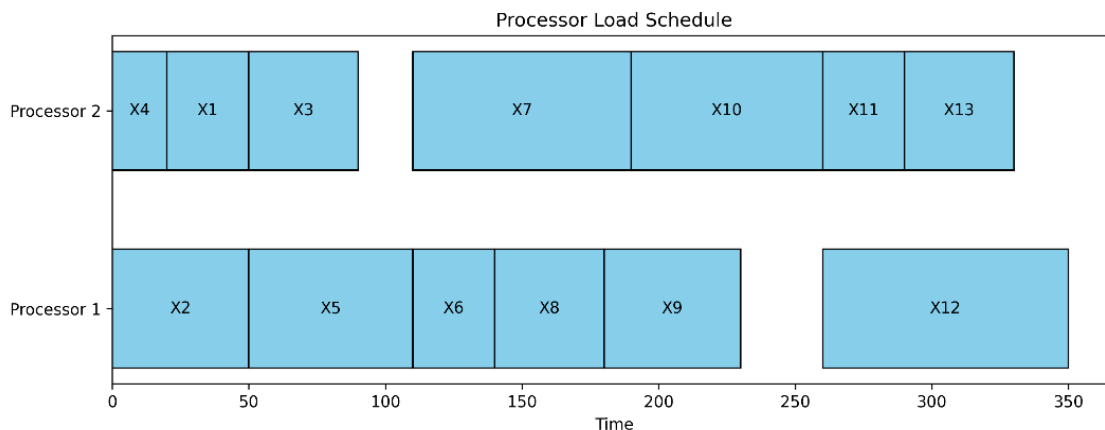


Рис 4. Граф с критическим путем

Критическое время $T_{кр} = 350$, что соответствует критическому пути $X2 \rightarrow X5 \rightarrow X7 \rightarrow X10 \rightarrow X12$.

Суммарное время всех задач $T_0 = 630$, а минимальное количество процессоров $T_0/T_{кр} = 2$

3. Построим график загрузки всех процессоров (рис 5), учитывая, что приступить к новой задаче можно только в том случае, если выполнены все работы, лежащие на пути к выбранной задаче и таблицу графика загрузки (рис. 6)



4. Рис 5. График загрузки процессов

	Time	Available	P1	P2
0	0-20	X1, X2, X3, X4	X2	X4
1	20-50	X1, X3	X2	X1
2	50-90	X5, X3	X5	X3
3	90-110		X5	idle
4	110-140	X6, X8, X7	X6	X7
5	140-180	X8, X9	X8	X7
6	180-190	X9	X9	X7
7	190-230	X10	X9	X10
8	230-260		idle	X10
9	260-290	X12, X11, X13	X12	X11
10	290-330	X13	X12	X13
11	330-350		X12	idle

Рис 6. Таблица графика загрузки процессов

Выводы.

В ходе выполнения лабораторной работы был изучен и реализован метод структурной оптимизации в многопроцессорной системе с общим доступом. На основе заданного графа зависимостей задач были определены ключевые характеристики модели, включая длительности выполнения узлов и структуру параллелизма.

Был вычислен критический путь, определяющий минимально возможное время выполнения всей совокупности задач. Значение критического времени составило $T_{кр} = 350$, что соответствует последовательности задач $X2 \rightarrow X5 \rightarrow X7 \rightarrow X10 \rightarrow X12$. Суммарное время всех операций составило $T_0 = 630$, что позволило определить минимально достаточное количество процессоров равное 2м

На основе этого значения было построено детализированное расписание загрузки процессоров, учитывающее все зависимости графа. Полученный график загрузки и таблица временных интервалов показали, что при использовании двух процессоров удаётся обеспечить выполнение всех работ в пределах критического времени $T_{кр}$, что подтверждает корректность рассчитанного значения $K_{пр}$.

Таким образом, были получены практические навыки моделирования и визуализации распределения нагрузки.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

```
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import math
from dataclasses import dataclass, field
from typing import List, Tuple

@dataclass
class TaskNode:
    name: str
    layer: int
    duration: int
    edges: List[str] = field(default_factory=list)
    index_in_layer: int = 0

def create_graph(tasks: List[TaskNode]) -> nx.DiGraph:
    G = nx.DiGraph()
    for task in tasks:
        G.add_node(
            task.name,
            layer=task.layer,
            duration=task.duration,
            index_in_layer=task.index_in_layer,
            C=task.duration
        )
        for target in task.edges:
            G.add_edge(task.name, target)
    return G

def compute_Cv(G):
    top_order = list(nx.topological_sort(G))
    for node in top_order:
        if G.in_degree(node) != 0:
            G.nodes[node]['C'] += max(G.nodes[pred]['C'] for pred
in G.predecessors(node))

def compute_critical_path(G: nx.DiGraph) -> List[str]:
    end_node = max(G.nodes, key=lambda n: G.nodes[n]['C'])
    path = [end_node]
    while G.in_degree(path[0]) > 0:
        preds = list(G.predecessors(path[0]))
        prev = max(preds, key=lambda p: G.nodes[p]['C'])
        path.insert(0, prev)
    return path

def show_graph(G: nx.DiGraph, critical_path: List[str] = [],
filename: str = "task_graph.png") -> None:
    for node in G.nodes():
        label = f"{node}\nti={G.nodes[node]['duration']}"
        if critical_path:
            label += f"\nC={G.nodes[node]['C']}"
        G.nodes[node]['label'] = label
```

```

        node_colors = ['red' if node in critical_path else 'lightblue'
for node in G.nodes()]
        pos = {n: (G.nodes[n]['layer'], -G.nodes[n]['index_in_layer'])
for n in G.nodes()}
        labels = nx.get_node_attributes(G, 'label')

        plt.figure(figsize=(8, 5))
        nx.draw(G, pos, with_labels=True, labels=labels,
node_size=3000, node_color=node_colors, arrowsize=20)
        plt.savefig(filename, format="png", dpi=300)
        plt.close()

def show_schedule(schedule, K_pr: int, filename: str =
"schedule.png") -> None:
    fig, ax = plt.subplots(figsize=(10, 2 + K_pr))
    for i, proc_tasks in enumerate(schedule):
        for task_info in proc_tasks:
            task_name = task_info['task']
            start = task_info['start']
            end = task_info['end']
            ax.barh(i, end - start, left=start, height=0.6,
color='skyblue', edgecolor='black')
            ax.text(start + (end - start)/2, i, task_name,
ha='center', va='center', color='black')

        ax.set_yticks(range(K_pr))
        ax.set_yticklabels([f"Processor {i + 1}" for i in range(K_pr)])
        ax.set_xlabel("Time")
        ax.set_title("Processor Load Schedule")
        plt.tight_layout()
        plt.savefig(filename, format="png", dpi=300)
        plt.close()

def build_schedule(G: nx.DiGraph, K_pr: int) -> Tuple[List, List]:
    remaining = {n: G.nodes[n]['duration'] for n in G.nodes()}
    completed = set()
    current_time = 0
    schedule = [[] for _ in range(K_pr)] # список задач для
каждого процессора
    schedule_steps = []

    processors = [None] * K_pr # текущие задачи на процессорах
    start_times = [None] * K_pr

    while len(completed) < len(G.nodes()):
        available_tasks = [
            n for n in G.nodes
            if n not in completed
            and n not in processors
            and all(pred in completed for pred in
G.predecessors(n))
        ]
        available_tasks_start = available_tasks.copy()

        # Назначаем доступные задачи на свободные процессоры
        for i in range(K_pr):

```

```

        if processors[i] is None and available_tasks:
            next_task = min(available_tasks, key=lambda n:
(G.nodes[n]['layer'], G.nodes[n]['index_in_layer']))
            available_tasks.remove(next_task)
            processors[i] = next_task
            start_times[i] = current_time

    # Определяем ближайшее завершение
    times = [remaining[p] for p in processors if p is not None]
    step = min(times)
    schedule_steps.append({
        'start': current_time,
        'end': current_time + step,
        'available': available_tasks_start.copy(),
        'processors': processors.copy()
    })
    current_time += step

    # Обновляем оставшееся время и записываем завершённые
задачи
    for i in range(K_pr):
        task = processors[i]
        if task is not None:
            remaining[task] -= step
            if remaining[task] <= 0:
                schedule[i].append({'task': task, 'start':
start_times[i], 'end': current_time})
                completed.add(task)
                processors[i] = None
                start_times[i] = None

    return schedule, schedule_steps

def print_schedule_df(schedule_steps, K_pr):
    rows = []
    for step in schedule_steps:
        row = {
            "Time": f"{step['start']}-{step['end']}",
            "Available": ", ".join(step['available']),
        }
        for i in range(K_pr):
            row[f"P{i+1}"] = step['processors'][i] if
step['processors'][i] is not None else "idle"
            rows.append(row)

    df = pd.DataFrame(rows)
    print(df)
    return df

def main():
    tasks = [
        TaskNode("X1", layer=1, duration=30, edges=["X6"],
index_in_layer=0),
        TaskNode("X2", layer=0, duration=50, edges=["X5"],
index_in_layer=0),
        TaskNode("X3", layer=1, duration=40, edges=["X8"],
index_in_layer=2),

```

```

        TaskNode("X4",      layer=0,      duration=20,      edges=["X5"],
index_in_layer=1),
        TaskNode("X5",      layer=1,      duration=60,      edges=["X6", "X7",
"X8"], index_in_layer=1),
        TaskNode("X6",      layer=2,      duration=30,      edges=["X9", "X10"],
index_in_layer=0),
        TaskNode("X7",      layer=2,      duration=80,      edges=["X10"],
index_in_layer=1),
        TaskNode("X8",      layer=2,      duration=40,      edges=["X10"],
index_in_layer=2),
        TaskNode("X9",      layer=3,      duration=50,      edges=["X12"],
index_in_layer=0),
        TaskNode("X10",     layer=3,      duration=70,      edges=["X11", "X12",
"X13"], index_in_layer=1),
        TaskNode("X11",      layer=4,      duration=30,      edges=[],
index_in_layer=1),
        TaskNode("X12",      layer=4,      duration=90,      edges=[],
index_in_layer=0),
        TaskNode("X13",      layer=4,      duration=40,      edges=[],
index_in_layer=2),
    ]

```

```

G = create_graph(tasks)

```

```

show_graph(G)
compute_Cv(G)
critical_path = compute_critical_path(G)
show_graph(G, critical_path, "task_graph_critical.png")

```

```

T_cr = max(G.nodes[n]['C'] for n in G.nodes())
T_o = sum(G.nodes[n]['duration'] for n in G.nodes())
K_pr = math.ceil(T_o / T_cr)

```

```

print("Суммарное время всех задач T_o:", T_o)
print("Критическое время T_cr:", T_cr)
print("Минимальное количество процессоров K_pr:", K_pr)

```

```

schedule, schedule_steps = build_schedule(G, K_pr)
print_schedule_df(schedule_steps, K_pr)
show_schedule(schedule, K_pr)

```

```

if __name__ == "__main__":
    main()

```